Backend Hands-on Coding – Software Engineer Grade II (Python / Java Spring Boot)

Overview

Build a small **vazhipadu (temple offering) booking service** for a temple board in one GitHub repository (in your personal github account and the link has to be shared once the work is completed), across five short problems. Attempt all five problems. Ship each one as its own pull request.

A vazhipadu is a ritual offering that a devotee pays the temple to perform. The devotee pays a fixed fee, and the priest performs the ritual, usually in the name of a person and their birth date.

- **Stack:** pick one track and use it for all five problems.
  - **Python:** Python 3.11+, FastAPI or Django.
  - **Java:** Java 17 or 21, Spring Boot 3.x.
- **Database:** any relational database. An embedded one is fine (H2 for Java, SQLite for Python). PostgreSQL is optional. No other infrastructure is needed.
- **Time limit:** 90 minutes for all five problems, including PRs, reviews and merges. Suggested split: setup 5 min, P1 20 min, P2 to P4 15 min each, final check 5 min.
- **What we assess:** working code that meets the acceptance criteria, edge cases, meaningful tests, clean code, and Git/PR discipline.

AI-assisted coding is expected. Use Claude Code, Copilot, Cursor or any agent you like. The problems describe *what* the business needs, not *how* to build it, and some details are deliberately left open. Making those decisions is part of the test. Write down every assumption you make in the PR. In the walkthrough you must explain every decision and any place where you corrected the AI.

Submission workflow

Work the way you would on a real team: each problem is one branch, one PR, one review and one merge, in order (P1 to P4).

1. Create a **public** GitHub repo named `tdb-vazhipadu-backend`. On `main`, add a README and the base project.
2. For each problem, create a branch from the latest `main`, for example `feature/p1-offerings`.
3. Commit in small, meaningful steps, such as `Add offering catalogue endpoints` or `Compute order total on server`. Avoid commits like `fix`, `changes`, `wip` or `final`. Do not commit directly to `main`.
4. Open a PR into `main` using the template below.

5. **Review the PR before merging.** Read the full diff as a reviewer. Leave at least one review comment on a specific line, either your own or one from an AI review tool. Address it with a follow-up commit or a short reply explaining why no change is needed.
6. Merge the PR (squash or merge commit, both fine), then pull `main` before starting the next problem.

PR description template

Problem
What this PR solves (1–2 lines).

Assumptions
Gaps in the problem statement you had to fill, and what you decided.

Decisions & trade-offs
What you chose, and why over the alternatives.

How to test
Commands, sample requests, test files.

The problems

P1 – Offerings and Orders API

Devotees book offerings (vazhipadu) such as Pushpanjali or Archana at a temple, for a chosen date.

Requirements

- Manage a temple's offerings: name, price and an active flag.
- A devotee places an order for one or more offerings, each with a quantity, for a date. The order records the devotee's name, phone number and birth star (nakshatra).
- The server calculates the order total.
- Get an order by id.
- Reject invalid input with a clear error. Use one consistent JSON error format for all errors.

Acceptance criteria

- An order for 2 Pushpanjali at ₹30 and 1 Archana at ₹50 has a total of ₹110.
- An order for an inactive offering is rejected.
- At least one automated test.

P2 – Order Search

Temple staff look up orders all day at the counter.

Requirements

- List orders for a temple, filtered by date and by status, and searchable by the devotee's phone number.
- The list can grow to hundreds of thousands of orders. Return it in pages, in a stable order.
- Use database migrations for your schema, and make sure the common searches don't scan the whole table.

Acceptance criteria

- Searching by phone number returns only that devotee's orders.
- Requesting page 2 returns the next set of results, with no repeats or gaps from page 1.

P3 – Daily Limits

Some offerings can only be performed a fixed number of times a day, for example 50 Udayasthamana Poojas.

Requirements

- An offering can have an optional daily limit.
- An order must be rejected if it would take the offering over its daily limit for that date.
- Online and counter orders are placed at the same time, so the limit must hold under simultaneous requests.

Acceptance criteria

- With a daily limit of 5 and 4 already booked, an order for 2 is rejected and an order for 1 succeeds.
- An automated test fires 20 simultaneous orders for 1 each against a daily limit of 5. Exactly 5 succeed.

P4 – Cancellation and Daily Collection

Requirements

- Staff can cancel an order. A cancelled order frees its daily limit.
- Add a daily collection report for a temple and date: the total amount and the count per offering.
- Finance compares this report with their books, so the totals must match to the paisa. Days are counted in Indian Standard Time.

Acceptance criteria

- A cancelled order is not counted in the report, and its daily limit becomes available again.
- An order placed at 00:15 IST on a date is counted on that date.

- The report total equals the sum of its offerings' totals.

Submission checklist

- Public repo `tdb-vazhipadu-backend`, with a README
- 4 branches and 4 PRs, each reviewed (at least one line comment addressed) and merged into `main`
- No direct commits to `main` after the initial setup
- Every PR uses the description template, including Assumptions
- The README explains how to run the app and the tests, and links all 4 PRs
- All tests pass
- No secrets committed
- Code and PRs done within 90 minutes
- You can explain every file in the repo without AI help